

UNIVERSIDADE FEDERAL DE MINAS GERAIS
DEPARTAMENTO DE ENGENHARIA ELETRÔNICA



ELT127– TURMA TN

SISTEMA DE INSPEÇÃO DE TÚNEIS
ETAPA FINAL

Lara Strutz Carvalho – 2023039325

Rian Rero Lopes Jericó Vieira – 2023039473

22 de junho de 2026

Sistema de Inspeção de Túneis

Etapa Final

Este projeto consiste num sistema de controle, simulação e operação remota de um robô autônomo focado na inspeção da integridade estrutural em túneis, desenvolvido como Trabalho Final da disciplina de Automação em Tempo Real (ATR). O sistema recorre a uma arquitetura híbrida que combina um núcleo crítico de tempo real programado em C++, responsável pela gestão de multitarefas, sincronização e controlo PID, com subsistemas periféricos criados em Python, que ficam encarregues da simulação física, interface gráfica e visão computacional, estando todos estes componentes perfeitamente integrados e a comunicar através do protocolo MQTT.

Autores:

Lara Strutz Carvalho – 2023039325

Rian Rero Lopes Jericó Vieira –
2023039473

Prof. Leandro Freitas de Abreu

22 de junho de 2026

Conteúdo

1	Introdução	1
1.1	Objetivos	2
2	Proposta de Arquitetura	2
2.1	Comunicação por Mensageria Assíncrona (MQTT)	3
2.2	Estrutura de Multitarefas e o Módulo de Tempo Real	4
2.3	Ferramentas de Sincronização e Exclusão Mútua	6
2.3.1	Contexto Compartilhado Seguro (Shared Context)	6
2.3.2	Filas Seguras para Múltiplas Threads (ThreadSafeQueue)	7
2.4	Controle de Temporização Rígida (Lógica Sleep Until)	8
2.5	TaskTimingLogger	9
2.6	Lógica de Detecção e Inspeção de Anomalias	10
2.7	Simulação do Sensor Inercial (IMU) e Navegação em Declives	13
2.8	Kernel de Tempo Real	14
3	Resultados de Testes Realizados	14
3.1	Interface de Operação, Telemetria e Controle Manual	14
3.2	Navegação Dinâmica e Adaptação a Variações de Inclinação (IMU)	16
3.3	Inspeção e Reação a Anomalias Estruturais	17
4	Análise de Capacidade e Escalabilidade do Sistema	19
4.1	Fundamentos Teóricos: Escalonabilidade por Taxa Monotônica	19
4.1.1	Condição Suficiente de Escalonabilidade	20
4.1.2	Análise Exata de Tempo de Resposta	20
4.2	Extração de Parâmetros a partir dos Dados Medidos	21
4.3	Determinação do Menor Período Admissível	21
4.4	Validação Experimental: Análise de Jitter e Cumprimento de Prazos	23
4.4.1	Experimento 1: Kernel Padrão, Sem Estresse	23
4.4.2	Experimento 2: Kernel Padrão, Com Estresse	25
4.4.3	Experimento 3: Kernel de Tempo Real (PREEMPT_RT), Com Estresse	27
4.4.4	Experimento 4: Kernel de Tempo Real, Sem Estresse (Referência RT)	29
4.5	Síntese Comparativa dos Resultados	30

5	Conclusões Finais	31
6	Atividades Extras	32
6.1	Análise com Kernel de Tempo Real (<code>PREEMPT_RT</code>)	32
6.2	Sensor IMU Simulado e Navegação em Declives	33
6.2.1	Modelagem do Terreno e Geração do Sinal IMU	33
6.2.2	Compensação Trigonométrica na Malha de Controle	33
6.3	Inspeção Visual com YOLOv8	34
6.3.1	Geração Sintética de Quadros (<code>SyntheticFrameGenerator</code>)	34
6.3.2	Inferência e Mecanismo de Fallback	34
6.3.3	Visualização: Bounding Box Verde no Simulador	35
6.3.4	Decisão de Projeto: Separação entre Detecção e Classificação	36
7	Apêndice	37

1 Introdução

A inspeção regular de túneis e estruturas civis é uma atividade crítica para garantir a segurança pública e prevenir acidentes catastróficos. Tradicionalmente, esse processo é realizado de forma manual ou com equipamentos rudimentares, o que se mostra não apenas custoso e demorado, mas também perigoso para os operadores envolvidos, dado o risco de desabamentos e ambientes insalubres. Diante desse cenário, o uso de robôs autônomos equipados com inteligência incorporada (*embodied intelligence*) apresenta-se como uma alternativa viável, permitindo que esses sistemas interajam com o ambiente em tempo real para otimizar a detecção de falhas.

Este trabalho aborda o desenvolvimento de um sistema de inspeção da integridade estrutural aplicado a túneis. O foco do sistema é mapear o perfil do teto do túnel e processar as imagens do ambiente para detectar anomalias superficiais (como buracos ou saliências) que podem indicar riscos estruturais iminentes. Para viabilizar e simplificar a implementação analítica, a cinemática e a física do robô foram modeladas em um plano 2D com vista lateral.



Figura 1: Ilustração conceitual de um robô móvel realizando o escaneamento do teto de um túnel para detecção de anomalias superficiais.

O código-fonte do projeto se encontra em [1] e o vídeo demonstrando o funcionamento do sistema está em [2].

1.1 Objetivos

O objetivo principal deste projeto é aplicar os conceitos e técnicas estudados na disciplina de Automação em Tempo Real (ATR) na implementação de um sistema completo de inspeção automática, contemplando tanto a simulação de sensores do robô quanto o núcleo de controle embarcado operando com restrições temporais.

Para alcançar este propósito, os seguintes objetivos específicos foram definidos:

- Desenvolver uma arquitetura de software híbrida, empregando um núcleo crítico em C++ responsável pelo controle em tempo real (*RTOS-like*), e subsistemas periféricos em Python responsáveis pela simulação física e percepção visual.
- Implementar uma malha de controle de malha fechada (PID) para manter a distância correta e segura entre o robô e o teto do túnel, utilizando dados simulados de sensores LIDAR.
- Garantir a intercomunicação e o tráfego de dados assíncrono entre os módulos e linguagens (C++ e Python) utilizando o protocolo de mensageria MQTT.
- Integrar um modelo de inteligência artificial (YOLOv8) para análise contínua das imagens da câmera e classificação de anomalias na estrutura física.
- Estabelecer mecanismos seguros de sincronização de tarefas no núcleo C++ (usando *mutexes*, *condition variables* e filas *thread-safe*) para evitar condições de corrida e garantir previsibilidade no tempo de resposta.

2 Proposta de Arquitetura

O sistema autônomo de inspeção de túneis foi desenvolvido utilizando uma abordagem arquitetural híbrida e modular, refinada a partir da primeira etapa do projeto. A arquitetura visa conciliar a necessidade de determinismo temporal e alta velocidade no laço de controle crítico com a flexibilidade de ferramentas modernas de inteligência artificial e interfaces gráficas.

A estrutura é dividida fundamentalmente em dois domínios: o **Core** C++, responsável pelo gerenciamento de tarefas de tempo real, cálculo de

distâncias, odometria e controle de malha fechada; e os **Periféricos Python**, encarregados da simulação física do ambiente do túnel, interface de supervisão (GUI) e análise visual por redes neurais convolucionais (YOLOv8).

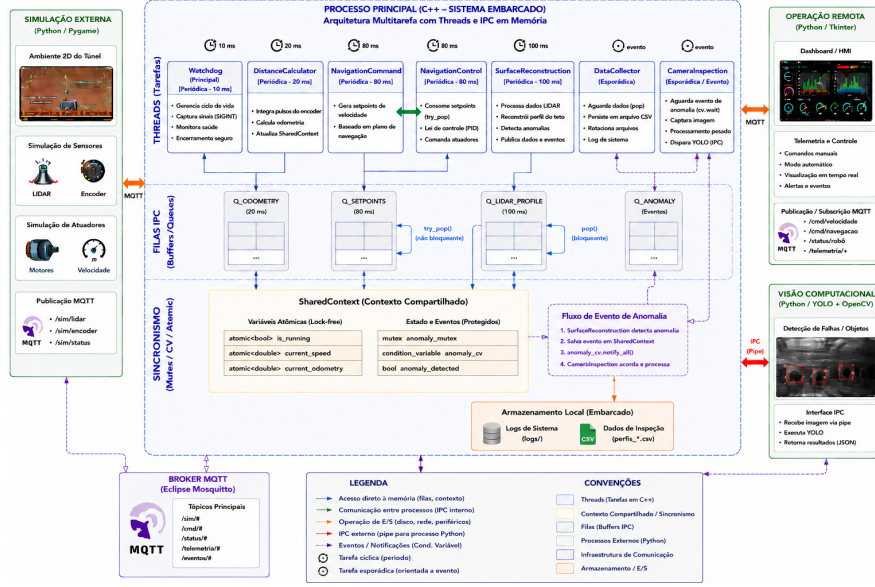


Figura 2: Diagrama de arquitetura híbrida do sistema, destacando o fluxo de dados entre o Core C++ e os componentes em Python via protocolo MQTT.

2.1 Comunicação por Mensageria Assíncrona (MQTT)

A integração entre os processos isolados desenvolvidos em C++ e Python baseia-se em um barramento de dados centralizado operando sob o protocolo MQTT (*Message Queuing Telemetry Transport*). Essa escolha de projeto confere desacoplamento espacial e temporal aos módulos.

Para a implementação do *Broker* central, optou-se pelo **Eclipse Mosquitto**, uma solução leve, de código aberto e altamente eficiente para o tráfego de mensagens locais. Visando a robustez do sistema, especialmente para as mensagens críticas da malha de controle e dados de odometria, a comunicação foi configurada para operar com **QoS 2 (Quality of Service nível 2 - Exactly once)**. Esse nível de qualidade de serviço assegura, através de um *handshake* de quatro vias, que cada mensagem seja entregue ao destinatário exatamente uma vez, mitigando por completo os riscos de

perdas ou duplicações de pacotes que poderiam desestabilizar o controlador PID ou gerar perdas de eventos no sistema de detecção.

O *Broker* Mosquitto gerencia tópicos específicos, tais como:

- **telemetria/robo**: Tráfego contínuo contendo a posição horizontal (X), vertical (Y), ângulo de inclinação (via sensor IMU simulado) e velocidade linear do robô.
- **sensores/lidar**: Distâncias medidas em relação ao teto e às paredes do túnel.
- **controle/comando**: Sinais de atuação gerados pelo controlador PID (e.g., velocidade de subida ou tração).
- **camera/deteccao**: Eventos de anomalia estrutural identificados pelo módulo de visão computacional.

O componente `MqttBridge` no Core C++ atua como uma tarefa gateway, convertendo as cargas úteis brutas vindas da rede em estruturas de dados nativas do sistema e vice-versa, operando de forma assíncrona para não bloquear as tarefas de tempo real.

2.2 Estrutura de Multitarefas e o Módulo de Tempo Real

No interior do núcleo C++, o gerenciamento de tarefas emula o comportamento de um Sistema Operacional de Tempo Real (*RTOS*). Todas as tarefas de controle e processamento estendem uma interface comum abstrata denominada `ITask`(Figura 3).

```

/**
 * @file ITask.hpp
 * @brief Definição da interface base para todas as tarefas cíclicas.
 */
#pragma once

...

namespace tasks {
/**
 * @class ITask
 * @brief Interface padrão que todas as tarefas do robô devem implementar.
 */
...
class ITask {
public:
    virtual ~ITask() = default;

    /**
     * @brief Método principal onde o loop infinito da thread deve ser implementado.
     */
    virtual void run() = 0;
};
} // namespace tasks

```

Figura 3: Modelagem e estrutura de métodos da interface base `ITask` utilizada para a padronização das tarefas críticas do sistema.

Cada tarefa é instanciada e executada em sua própria thread nativa do POSIX (`std::thread`) (Figura 4), sendo categorizadas em cíclicas (periódicas) e orientadas a eventos:

1. Tarefas Periódicas (Cíclicas):

- `DistanceCalc` (Período: 20 ms): Realiza a filtragem e conversão analítica dos sinais brutos dos sensores de distância.
- `NavigationControl` (Período: 80 ms): Executa o algoritmo PID contínuo para regular a suspensão ou distância vertical em relação ao teto irregular.
- `NavigationCommand` (Período: 80 ms): Computa a odometria baseada nos encoders e envia as diretrizes de movimentação horizontal.
- `SurfaceRecon` (Período: 100 ms): Reconstrói o perfil do teto agregando as leituras geométricas no espaço cartesiano do túnel.

2. Tarefas Orientadas a Eventos:

- `DataIMU`: Despertada assincronamente mediante a chegada de novas leituras do sensor de inclinação para correções rápidas de atitude física.

- **CameraInspection:** Ativada sob demanda quando o sistema de visão notifica uma anomalia estrutural crítica na superfície.

```
// Lançamento das Threads
std::vector<std::thread> thread_pool;
thread_pool.emplace_back([task_reconstruction]() { task_reconstruction->run(); }); // [0]
thread_pool.emplace_back([task_camera]() { task_camera->run(); }); // [1]
thread_pool.emplace_back([task_nav_cmd]() { task_nav_cmd->run(); }); // [2]
thread_pool.emplace_back([task_nav_ctrl]() { task_nav_ctrl->run(); }); // [3]
thread_pool.emplace_back([task_dist_calc]() { task_dist_calc->run(); }); // [4]
thread_pool.emplace_back([task_collector]() { task_collector->run(); }); // [5]
thread_pool.emplace_back([task_mqtt_bridge]() { task_mqtt_bridge->run(); }); // [6]
```

Figura 4: Trecho de código do ponto de entrada do sistema (`main.cpp`) ilustrando a instanciação e o lançamento das múltiplas *threads* concorrentes no *Core C++*. A utilização da classe nativa `std::thread` assegura o isolamento em nível de processador para cada tarefa periódica e para o *gateway* de mensageria (MQTT).

2.3 Ferramentas de Sincronização e Exclusão Mútua

Para garantir a previsibilidade e mitigar riscos clássicos de sistemas concorrentes, tais como condições de corrida (*race conditions*), inversão de prioridade e corrupção de memória, foram adotados mecanismos formais de sincronização baseados nas bibliotecas nativas de concorrência do C++.

2.3.1 Contexto Compartilhado Seguro (Shared Context)

As threads do sistema necessitam ler e atualizar variáveis de estado globais (como coordenadas atuais do robô e o estado da máquina de estados do sistema). Essa área de memória crítica foi encapsulada na classe `SharedContext`.

O acesso a qualquer recurso interno dessa classe é rigidamente controlado através de exclusão mútua empregando primitivas `std::mutex`. Para mitigar o risco de *deadlocks* causados por esquecimento de liberação de travas em ramificações do código ou exceções, utilizou-se o padrão RAII (*Resource Acquisition Is Initialization*) por meio do adaptador `std::lock_guard` ou `std::unique_lock`, conforme ilustrado na Figura 5

```

void SharedContext::triggerAnomaly() {
    std::lock_guard<std::mutex> lock(anomaly_mutex_);
    anomaly_detected_ = true;
    anomaly_cv_.notify_all();
}

void SharedContext::resetAnomaly() {
    std::lock_guard<std::mutex> lock(anomaly_mutex_);
    anomaly_detected_ = false;
}

void SharedContext::waitForAnomaly() {
    std::unique_lock<std::mutex> lock(anomaly_mutex_);
    anomaly_cv_.wait(lock, [this]() { return anomaly_detected_ || !is_running; });
}

bool SharedContext::isAnomalyActive() {
    std::lock_guard<std::mutex> lock(anomaly_mutex_);
    return anomaly_detected_;
}

```

Figura 5: Implementação dos métodos de sincronização na classe `SharedContext`. O código ilustra o uso de exclusão mútua (`std::mutex`) e variáveis de condição (`std::condition_variable`) para a sinalização assíncrona e segura (*thread-safe*) de eventos de anomalia, eliminando a espera ativa (*busy-waiting*).

2.3.2 Filas Seguras para Múltiplas Threads (`ThreadSafeQueue`)

Para a comunicação inter-threads orientada a dados (mensagens de log, armazenamento local de dados no `DataCollector` e repasse de eventos de detecção), a utilização de variáveis globais simples é inadequada devido à necessidade de enfileiramento. Desenvolveu-se, portanto, uma estrutura de dados genérica do tipo FIFO denominada `ThreadSafeQueue<T>`.

Essa estrutura utiliza internamente uma combinação de `std::mutex` com variáveis de condição (`std::condition_variable`). Quando uma tarefa produtora insere um dado na fila (`push`), ela adquire a trava, insere o elemento na estrutura interna `std::queue` e invoca `notify_one()` na variável de condição. A tarefa consumidora (e.g., o coletor de dados), por sua vez, executa um método bloqueante (`wait_and_pop`). Se a fila estiver vazia, a thread consumidora é colocada em estado de espera pelo sistema operacional, liberando a CPU até que um novo dado seja postado, eliminando completamente o desperdício de ciclos de processamento por *busy waiting*.

```

template <typename T>
bool ThreadSafeQueue<T>::push(const T& item) {
    std::unique_lock<std::mutex> lock(mutex_);

    // Produtor dorme enquanto o buffer estiver cheio (count_ == capacity_).
    cv_.wait(lock, [this]() { return closed_ || count_ < capacity_; });

    if (closed_) {
        return false;
    }

    // Escreve na posição head_ e avança head_ circularmente.
    data_[head_] = item;
    head_ = (head_ + 1) % capacity_;
    count_++;

    lock.unlock();
    cv_.notify_one();
    return true;
}

```

Figura 6: Detalhamento do método de inserção (push) na estrutura ThreadSafeQueue. O uso integrado de `std::unique_lock` e variável de condição (`cv_.wait` e `cv_.notify_one`) garante o bloqueio seguro da tarefa produtora caso o *buffer* atinja sua capacidade máxima, sincronizando a troca de dados sem desperdício de CPU.

2.4 Controle de Temporização Rígida (Lógica Sleep Until)

Em automação de tempo real, garantir a periodicidade exata de execução de uma tarefa cíclica é vital para o correto funcionamento de filtros digitais e controladores dinâmicos (como o PID). A utilização de suspensões de tempo relativas (como a função `sleep_for`) introduz um erro acumulativo severo, visto que o tempo gasto no processamento interno da tarefa não é contabilizado, resultando em deriva temporal e alto *jitter*.

Para contornar esse problema, a lógica utiliza uma temporização absoluta baseada em pontos no tempo (*time points*). Utiliza-se o relógio monotônico do sistema (`std::chrono::steady_clock`), imune a ajustes manuais ou correções de rede do relógio de tempo real do computador de hardware.

O algoritmo estabelece o tempo inicial ideal de ativação e, ao fim de cada ciclo de execução, incrementa estritamente o período nominal da tarefa ao ponto temporal de referência. A thread é então colocada para dormir até

esse instante absoluto futuro via `std::this_thread::sleep_until`, o que absorve variações no tempo de processamento interno do código.

```
auto next_wakeup = std::chrono::steady_clock::now();  
const auto cycle_time = std::chrono::milliseconds(20);
```

(a) Inicialização do relógio monotônico (*steady_clock*) e definição do período de execução (ex: 20 ms) antes do ingresso no laço principal.

```
while (context_>is_running) {  
    auto scheduled = next_wakeup;  
    auto actual = std::chrono::steady_clock::now();  
    next_wakeup += cycle_time;
```

(b) Atualização do instante de despertar (`next_wakeup += cycle_time`). Ao calcular o próximo ciclo somando o período ao alvo absoluto anterior (e não ao instante atual de execução), o sistema elimina o acúmulo de atrasos (*drift*) causado pelo tempo de processamento das instruções da *thread*.

```
std::this_thread::sleep_until(next_wakeup);
```

(c) Suspensão da *thread* através da primitiva `sleep_until`, garantindo que a retomada ocorra exatamente no marco temporal absoluto planejado.

Figura 7: Fluxo completo da implementação da lógica de temporização absoluta no Core C++. A sequência demonstra o mecanismo utilizado para compensar dinamicamente o tempo de processamento interno da CPU e estabilizar a periodicidade das tarefas cíclicas.

2.5 TaskTimingLogger

Para viabilizar a análise de desempenho do sistema, o núcleo de tempo real foi instrumentado por meio da classe `TaskTimingLogger`. Esta estrutura é responsável por extrair e registrar os *timestamps* de alta resolução do relógio do sistema a cada ciclo de processamento. Especificamente, o mecanismo captura três marcos temporais críticos para cada tarefa: o instante nominal em que o despertar foi agendado (*scheduled*), o instante em que a execução

foi efetivamente despachada (*actual*) e o instante de sua respectiva conclusão (*exec_end*).

A geração contínua destes *traces* de execução fornece a base de dados para o cálculo de métricas críticas em sistemas de tempo real, como o *Jitter* e o Tempo de Resposta. Para garantir a rastreabilidade, todos os dados coletados são exportados para um arquivo estruturado no formato CSV. A partir desses registros, torna-se possível gerar gráficos de escalonamento (como diagramas de Gantt) e atestar o cumprimento estrito dos prazos (*deadlines*) durante toda a rotina de inspeção do robô, validando o determinismo e a estabilidade temporal da arquitetura implementada.

```
class TaskTimingLogger {
public:
    ...

    struct Record {
        const char* task;
        int period_ms;
        uint64_t cycle;
        uint64_t scheduled_ns;
        uint64_t actual_ns;
        uint64_t exec_end_ns;
    };

    static TaskTimingLogger& instance();

    void open(const std::string& path);
    void log(const Record& r);

    template <typename TimePoint>
    static uint64_t toNs(TimePoint tp) {
        return static_cast<uint64_t>{
            std::chrono::duration_cast<std::chrono::nanoseconds>(tp.time_since_epoch()).count()
        };
    }

private:
    TaskTimingLogger() = default;
    std::ofstream file_;
    std::mutex mutex_;
};

} // namespace core
```

Figura 8: Estrutura da classe `TaskTimingLogger`, desenvolvida para a monitorização de desempenho em tempo real das *threads*. A recolha contínua destas métricas é a base para a extração do tempo de resposta e do *jitter* analisados na avaliação de escalabilidade da arquitetura.

2.6 Lógica de Detecção e Inspeção de Anomalias

O processo de identificação de falhas estruturais no túnel e a subsequente inspeção visual exigem um alto nível de determinismo e sincronização. A

transição do robô de seu estado de cruzeiro para o modo de inspeção minuciosa desencadeia uma sequência coordenada de eventos que atravessa múltiplas *threads* no núcleo C++ e culmina no processamento periférico em Python, operando no seguinte fluxo cronológico:

1. **Detecção e Sinalização Interna:** A *thread* cíclica `SurfaceReconstruction` monitora ativamente as leituras verticais do LIDAR. Ao constatar que a variação da leitura em relação à distância nominal do teto (2,0m) supera o limiar configurável `threshold_anomaly_` (padrão: 0,4m de desvio), ela invoca o método `triggerAnomaly()` do `SharedContext`. Sob proteção de exclusão mútua, a variável de estado da anomalia é ativada e a primitiva `notify_all()` é disparada na variável de condição.
2. **Ajuste Dinâmico de Navegação:** Imediatamente após a sinalização no contexto global, a *thread* responsável pelos comandos de navegação (`NavigationCommand`) reconhece a alteração de estado em seu próximo ciclo de 80 ms. Para garantir estabilidade física e tempo hábil de captura de imagem, o sistema reduz a velocidade de translação do robô, alterando o *setpoint* de velocidade de cruzeiro (50%) para um modo de aproximação lenta (15%).
3. **Despertar da Inspeção:** Simultaneamente, a *thread* `CameraInspection`, que permanecia bloqueada passivamente em `waitForAnomaly()`, é despertada pelo sinal `notify_all()`. Ao retomar sua execução, ela publica o valor '1' em dois tópicos MQTT em sequência: primeiro em `state/inspection`, sinalizando à interface gráfica que uma inspeção está em andamento, e em seguida em `cmd/camera`, disparando o serviço de visão em Python.
4. **Geração de Quadro e Processamento Visual:** O módulo periférico em Python (`YoloInspectionService`) recebe a requisição via *Broker*. Ele utiliza a biblioteca OpenCV para sobrepor sinteticamente o defeito detectado no quadro gerado e inicia a inferência pelo modelo YOLOv8, cujo resultado é publicado no tópico `telemetry/yolo`.
5. **Finalização e Retomada de Cruzeiro:** Para viabilizar a conclusão da análise externa, a *thread* `CameraInspection` no C++ executa um *busy wait* ativo de 1500 ms. Ao término desse intervalo, publica '0' nos tópicos `cmd/camera` e `state/inspection`, sinalizando o fim do

evento à câmera e à interface. A *thread* então aguarda passivamente, em laço com `sleep_for` de 50 ms, até que a leitura do LIDAR normalize e `resetAnomaly()` seja invocado pela `SurfaceReconstruction`, autorizando o retorno à velocidade de cruzeiro.

```
double limite_variacao = std::abs(threshold_anomaly_ - kNominalCeilingDistanceM);

// Detecta mudança de estado da anomalia (só muda flag, não imprime ainda)
std::string deferred_log_msg;
core::TerminalPrinter::Level deferred_log_level{};
bool has_deferred_log = false;

if (variacao >= limite_variacao) {
    if (!context_>isAnomalyActive()) {
        deferred_log_msg = "ALERTA: Variação estrutural (" +
                           std::to_string(simulated_lidar_y) +
                           "m)! Disparando flag global.";
        deferred_log_level = core::TerminalPrinter::Level::Warning;
        has_deferred_log = true;
        context_>triggerAnomaly();
    }
} else {
    if (context_>isAnomalyActive()) {
        deferred_log_msg = "Superfície normalizada. Desativando flag.";
        deferred_log_level = core::TerminalPrinter::Level::Info;
        has_deferred_log = true;
        context_>resetAnomaly();
    }
}
```

Figura 9: Trecho de código da tarefa `SurfaceReconstruction` ilustrando o início do fluxo de detecção. Ao identificar uma anomalia geométrica através do LIDAR, o sistema interage com o `SharedContext` para disparar a sinalização global em tempo real.

É fundamental destacar uma premissa de projeto adotada nesta arquitetura: a classificação rigorosa do tipo de anomalia (buraco ou saliência) não é delegada à rede neural YOLOv8. Uma vez que o modelo empregado utiliza pesos pré-treinados genéricos e não passou por um *fine-tuning* especializado em estruturas de engenharia civil, depender dele para a validação do tipo de falha introduziria erros na identificação.

Para assegurar a total confiabilidade do sistema, a detecção primária e o diagnóstico permanecem estritamente determinísticos, baseados na magnitude da variação captada pelos sensores. O módulo de visão atua como uma ferramenta de apoio visual e demonstração do acionamento coordenado da câmera, sem corromper a malha crítica de controle de segurança do RTOS.

2.7 Simulação do Sensor Inercial (IMU) e Navegação em Declives

Como requisito avançado para aumentar a robustez e o realismo do robô de inspeção, foi implementado um módulo dedicado à simulação de um Sensor Inercial (IMU - *Inertial Measurement Unit*). Este acréscimo permite que o sistema ultrapasse a restrição analítica de operar apenas em um plano perfeitamente horizontal, viabilizando a navegação autônoma em túneis com diferentes graus de declive.

No ambiente de simulação em Python, o terreno do túnel foi modelado com variações de inclinação. O simulador calcula a atitude física do robô, especificamente o ângulo de arfagem (*pitch*, denotado por θ), e publica essa métrica continuamente no barramento MQTT.

No núcleo de tempo real em C++, o tratamento desse dado é de alta criticidade, pois influencia diretamente a odometria e a malha de controle. Devido à alta frequência de concorrência, com a *thread* do MQTT escrevendo o dado e as tarefas `DistanceCalc` e `NavigationControl` lendo-o simultaneamente, o ângulo de inclinação foi instanciado no `SharedContext` utilizando a estrutura `std::atomic<double>` (Figura 10).

```
// 2. Simula um túnel com declive real: a IMU mede o ângulo do piso.  
const double imu_degrees = std::atan(floorSlope(current_x)) * kRadiansToDegrees;  
context_->imu_degrees.store(imu_degrees);
```

Figura 10: Lógica de simulação e recepção das leituras do sensor inercial (IMU). O dado processado é armazenado diretamente em uma variável do tipo `std::atomic` no contexto global, viabilizando operações de leitura e escrita *lock-free* concorrentes de alto desempenho.

A adoção de tipos atômicos nativos do C++ garante a exclusão mútua em nível de *hardware* (através de instruções atômicas do próprio processador), dispensando completamente o uso de *mutexes* tradicionais para essa variável. Essa arquitetura *lock-free* elimina o risco de inversão de prioridade e minimiza o *overhead* de sincronização, assegurando que o sistema acesse a inclinação instantânea sem sofrer penalidades de latência no escalonamento.

Além da otimização computacional, os dados do IMU possuem um papel matemático vital na estabilidade do robô. Quando o veículo entra em um declive, o feixe do sensor LIDAR (que está fixo no referencial local do chassi) passa a medir uma distância diagonal em relação ao teto. Para evitar que o

controlador PID interprete essa diagonal como um erro de posicionamento vertical e atue de forma indesejada, a tarefa `DistanceCalc` aplica uma correção trigonométrica em tempo real. A distância bruta é projetada no eixo vertical global multiplicando-se a leitura pelo cosseno do ângulo de inclinação ($y_{real} = y_{sensor} \cdot \cos(\theta)$). Essa compensação dinâmica prova a integração bem-sucedida entre o sistema de percepção e a física de controle do RTOS.

2.8 Kernel de Tempo Real

O kernel de tempo real (`PREEMPT_RT`) é ativado por meio das chamadas POSIX `SCHED_FIFO` e `mlockall`, que são executadas no início do programa caso o processo possua permissões de superusuário. Sem essas permissões, o sistema ainda opera corretamente, porém sem as garantias de latência discutidas na Seção 4.

A política `SCHED_FIFO` (*First-In, First-Out* de prioridade fixa) é a política de tempo real mais direta do POSIX: uma thread com essa política e uma dada prioridade só é preemptada por outra thread com prioridade estritamente superior. As prioridades são atribuídas em conformidade com o teorema da Taxa Monotônica, com a tarefa de menor período recebendo a maior prioridade numérica. A chamada `mlockall(MCL_CURRENT | MCL_FUTURE)` trava todas as páginas de memória do processo na RAM física, eliminando falhas de página (*page faults*) durante a execução, que são uma fonte clássica de jitter não-determinístico em sistemas de tempo real.

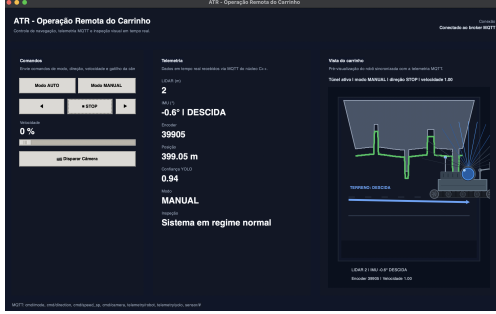
3 Resultados de Testes Realizados

Para validar a integridade, a robustez e o funcionamento global do sistema sob a arquitetura híbrida desenvolvida, foram executados testes funcionais visando comprovar a eficácia da malha de controle em tempo real, a confiabilidade da comunicação assíncrona (MQTT) e a resposta dinâmica do sistema a diferentes condições topográficas e falhas estruturais do túnel.

3.1 Interface de Operação, Telemetria e Controle Manual

O primeiro estágio de validação focou-se na integridade do tráfego de dados e no comportamento da Interface Gráfica de Supervisão (GUI). O sistema

exige que o estado interno do robô gerenciado pelo *Core C++* seja refletido instantaneamente para o operador.



(a) Telemetria com a velocidade do robô ajustada para zero.



(b) Telemetria refletindo o comando de deslocamento para a esquerda.



(c) Ajuste manual do *setpoint* de velocidade.



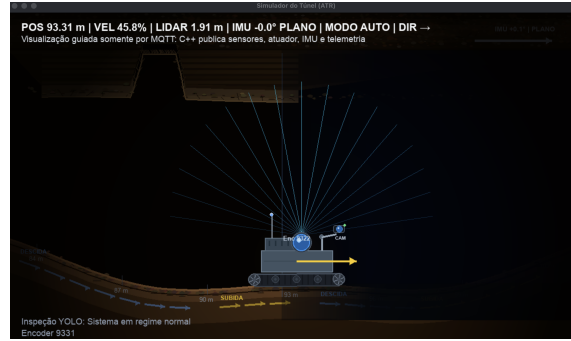
(d) Disparo da câmera no modo manual.

Figura 11: Painéis da Interface de Operação (GUI). Na linha superior, a telemetria evidencia a resposta do sistema a diferentes estados operacionais (repouso e translação), atualizando em tempo real os parâmetros vitais (Modo, Velocidade, *Pitch*, LIDAR e coordenadas cartesianas). Na linha inferior, o painel de intervenção manual que permite ao operador alterar *setpoints* de velocidade e disparar comandos assíncronos para a câmera via MQTT.

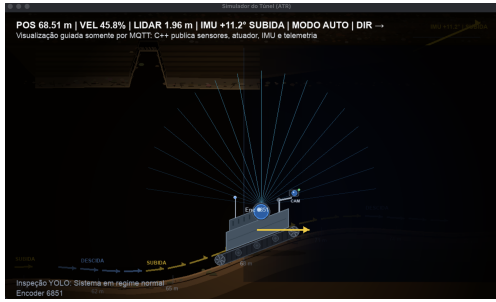
Como observado na Figura 11, o barramento de mensageria operou de forma estável. As variáveis atômicas e de estado (como o *Pitch* e as leituras do LIDAR) foram transmitidas sem perda de pacotes, e os botões de intervenção manual sobrepuseram com sucesso os comandos automáticos, validando a lógica bidirecional do *MqttBridge*.

3.2 Navegação Dinâmica e Adaptação a Variações de Inclinação (IMU)

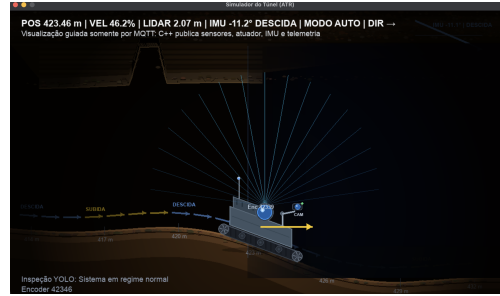
O segundo cenário testou a estabilidade do robô no simulador físico em 2D. O teste incluiu a navegação em velocidade de cruzeiro num plano retilíneo horizontal, bem como a transição para terrenos acidentados, englobando situações de aclave (subida) e declive (descida).



(a) Navegação estável em trecho retilíneo horizontal.



(b) Veículo enfrentando aclave (subida).



(c) Veículo enfrentando declive (descida).

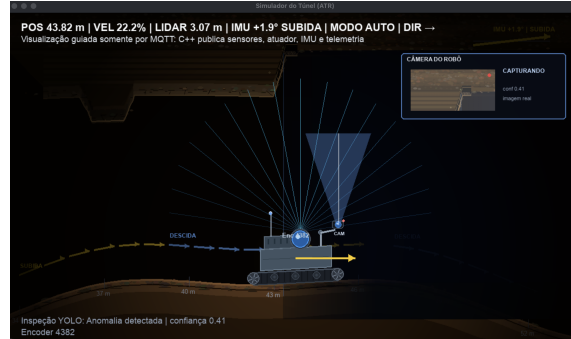
Figura 12: Renderização do simulador ilustrando a adaptação física do robô a diferentes topografias do túnel. Em todos os cenários, a leitura do IMU permite aplicar a compensação trigonométrica na leitura do LIDAR, impedindo atuações erráticas da suspensão.

A resposta do controlador PID mostrou-se robusta. Tanto na subida (Figura 12b) quanto na descida (Figura 12c), a integração com o sensor inercial (IMU) simulado evitou que a inclinação do chassi causasse uma falsa leitura de

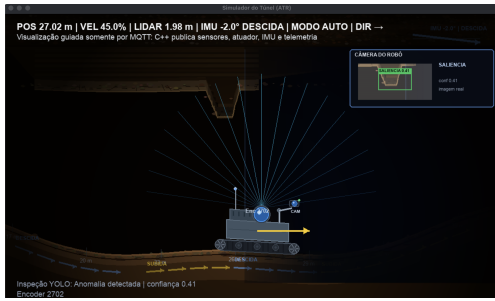
distanciamento do teto. A compensação matemática da hipotenusa garantiu que o eixo vertical de referência fosse mantido rigorosamente constante, atestando o determinismo do subsistema de odometria e cálculo de distância.

3.3 Inspeção e Reação a Anomalias Estruturais

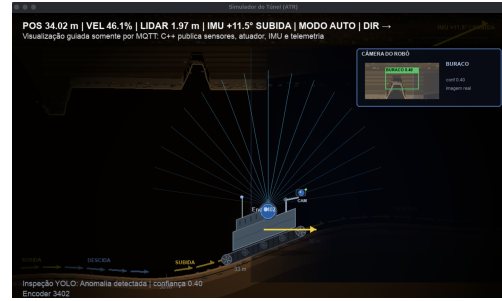
O ensaio mais crítico avaliou a coordenação entre a detecção analítica de uma deformação civil (feita pelo C++) e a subsequente captura e classificação visual (feita pelo Python). Foram simuladas geometrias de saliências e buracos no teto do túnel.



(a) Módulo de visão acionado (*Capturando...*).



(b) Saliência detectada e processada pela interface.



(c) Buraco estrutural identificado e demarcado.

Figura 13: Sequência de acionamento do modo de inspeção. O sistema detecta a anomalia via LIDAR, altera seu modo operacional (reduzindo a velocidade) e aciona o módulo de visão. Após a renderização sintética, diferentes manifestações patológicas (como buracos e saliências) são processadas e suas demarcações (*bounding boxes*) são publicadas de volta para a interface do utilizador.

Os resultados (Figura 13) comprovam o sucesso do fluxo de sincronização abordado na arquitetura. No exato instante em que o degrau ultrapassou a tolerância geométrica configurada na tarefa **SurfaceReconstruction**, a interrupção ocorreu: o robô diminuiu sua velocidade e a *flag* global despertou o serviço de câmera. A captura da imagem da anomalia e a classificação operaram corretamente dentro do *busy wait* alocado de 1500 ms, retornando ao estado nominal logo em seguida, sem qualquer violação de *deadline*.

4 Análise de Capacidade e Escalabilidade do Sistema

Esta seção apresenta a análise quantitativa do comportamento temporal do núcleo de tempo real implementado, respondendo à questão central exigida pelo enunciado do trabalho: dado o sistema computacional utilizado, qual o menor período admissível para as tarefas cíclicas que ainda garante a escalonabilidade do sistema?

A resposta é construída em três etapas. Primeiro, apresentam-se os fundamentos teóricos que embasam a análise. Em seguida, o modelo é aplicado aos dados reais coletados durante a execução. Por fim, os resultados são validados experimentalmente por meio de testes de estresse sob o kernel padrão do Linux e sob o kernel de tempo real (PREEMPT_RT).

4.1 Fundamentos Teóricos: Escalonabilidade por Taxa Monotônica

A política de escalonamento adotada neste projeto é a **Taxa Monotônica** (*Rate-Monotonic*, RM), o algoritmo ótimo para conjuntos de tarefas periódicas com prazos iguais aos períodos, conforme demonstrado por Liu e Layland [3]. Nesse esquema, a prioridade de escalonamento de cada tarefa é inversamente proporcional ao seu período: quanto menor o período, maior a prioridade. A implementação em `main.cpp` (Figura 14) reflete exatamente essa política, atribuindo prioridades POSIX `SCHED_FIFO` em ordem decrescente de período.

```
// — RT Linux: prioridades Rate-Monotonic (período menor = prioridade maior) —
// SCHED_FIFO prio [1,99]: DistanceCalc(20ms)>NavCmd/Ctrl(80ms)>SurfaceRecon(100ms)
setThreadRT(thread_pool[4], 50); // DistanceCalculator - 20ms
setThreadRT(thread_pool[2], 40); // NavigationCommand - 80ms
setThreadRT(thread_pool[3], 39); // NavigationControl - 80ms
setThreadRT(thread_pool[0], 30); // SurfaceReconstruction- 100ms
setThreadRT(thread_pool[5], 20); // DataCollector - evento
setThreadRT(thread_pool[1], 15); // CameraInspection - evento
setThreadRT(thread_pool[6], 10); // MqttBridge - 200ms
```

Figura 14: Atribuição estática de prioridades às *threads* no *Core C++* de acordo com a política *Rate-Monotonic* (RM). Utilizando o agendador *SCHED_FIFO* do padrão POSIX de tempo real, a tarefa com o menor período (*DistanceCalculator*, 20 ms) recebe a maior prioridade (50), enquanto tarefas com períodos maiores ou baseadas em eventos recebem prioridades progressivamente menores.

4.1.1 Condição Suficiente de Escalonabilidade

Para um conjunto de n tarefas independentes, a condição suficiente de escalabilidade da RM é dada pela seguinte desigualdade sobre a utilização total da CPU:

$$U = \sum_{i=1}^n \frac{C_i}{T_i} \leq n \left(2^{1/n} - 1 \right) \quad (1)$$

onde C_i é o pior tempo de execução (*Worst-Case Execution Time*, WCET) da tarefa i , e T_i é seu período. O limite do lado direito converge para $\ln(2) \approx 0,693$ quando $n \rightarrow \infty$. Essa condição é *suficiente*, não necessária: o sistema pode ser escalonável mesmo com $U > n(2^{1/n} - 1)$, mas a prova formal exige análise de tempo de resposta.

4.1.2 Análise Exata de Tempo de Resposta

O tempo de resposta no pior caso (*Worst-Case Response Time*, WCRT) para a tarefa i é obtido pela equação de ponto fixo proposta por Joseph e Pandya [4]:

$$R_i^{(k+1)} = C_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i^{(k)}}{T_j} \right\rceil C_j \quad (2)$$

onde $hp(i)$ é o conjunto de tarefas com prioridade estritamente superior à tarefa i . O cálculo parte de $R_i^{(0)} = C_i$ e itera até a convergência ($R_i^{(k+1)} = R_i^{(k)}$). O sistema é escalonável se e somente se $R_i \leq T_i$ para todas as tarefas.

4.2 Extração de Parâmetros a partir dos Dados Medidos

Os dados coletados pelo `TaskTimingLogger` durante uma execução normal (sem carga artificial e sem kernel de tempo real) fornecem os parâmetros necessários para a análise. O WCET de cada tarefa cíclica é extraído como o valor máximo de tempo de execução observado ao longo de todos os ciclos registrados, conforme a Tabela 1.

Tabela 1: Parâmetros extraídos dos logs de temporização para a análise de escalonabilidade (execução sem estresse, kernel padrão).

Tarefa	Período T_i	WCET C_i (medido)	Utilização C_i/T_i	Prioridade RM
DistanceCalc	20 ms	94,1 μs	0,00471	1 ^a (maior)
NavCommand	80 ms	69,3 μs	0,00087	2 ^a
NavControl	80 ms	94,7 μs	0,00118	3 ^a
SurfaceRecon	100 ms	47,4 μs	0,00047	4 ^a (menor)
Utilização total U			0,00723	

A utilização total medida de $U \approx 0,72\%$ é extraordinariamente baixa. O limite da RM para $n = 4$ tarefas é:

$$U_{RM}(4) = 4 \left(2^{1/4} - 1 \right) \approx 4 \times 0,1892 = 0,7568 \text{ (75,68\%)} \quad (3)$$

A utilização atual representa apenas 0,0095 do limite teórico, evidenciando que o sistema opera com uma folga de escalonamento ampla sob condições normais.

4.3 Determinação do Menor Período Admissível

A questão central é: mantendo a razão entre os períodos constante (20:80:80:100), qual o fator de escalonamento α mínimo que ainda torna o sistema inescalonável?

Se os períodos forem escalados por um fator α ($0 < \alpha < 1$), os novos períodos passam a ser $\alpha \cdot T_i$. Como os WCETs C_i permanecem constantes (são determinados pelo hardware), a nova utilização total é:

$$U(\alpha) = \sum_{i=1}^n \frac{C_i}{\alpha \cdot T_i} = \frac{U_0}{\alpha} \quad (4)$$

O sistema deixa de satisfazer a condição suficiente da RM quando $U(\alpha) > U_{RM}(n)$:

$$\alpha_{min,suf} = \frac{U_0}{U_{RM}(n)} = \frac{0,0000723}{0,7568} \approx 0,0000955 \quad (5)$$

Esse resultado significa que, em teoria, os períodos poderiam ser reduzidos por um fator de até $\approx 10.000\times$ antes que a condição suficiente fosse violada. Na prática, esse valor é dominado por restrições físicas do sistema operacional, como o quantum do escalonador e a latência de interrupção. Para determinar o menor período *praticamente* viável, aplica-se a análise de tempo de resposta (WCRT) com os parâmetros medidos.

Para a tarefa de maior prioridade (**DistanceCalc**, $C_1 = 94,1 \mu s$), por ser a tarefa com maior prioridade, ela não sofre preempção de nenhuma outra. Portanto, $R_1 = C_1 = 94,1 \mu s$. O menor período admissível para essa tarefa é o valor imediatamente acima do seu WCET:

$$T_{1,min} = R_1 = 94,1 \mu s \ (\approx 0,1 \text{ ms}) \quad (6)$$

Para as tarefas de prioridade inferior, o WCRT inclui a interferência das tarefas de maior prioridade. O menor período da tarefa de menor prioridade (**SurfaceRecon**) que satisfaz $R_4 \leq T_4$ é determinado pela iteração da equação (2). Com os parâmetros medidos, o WCRT de **SurfaceRecon** no sistema atual é:

$$R_4 = 47,4 + \left\lceil \frac{R_4}{20} \right\rceil \cdot 94,1 + \left\lceil \frac{R_4}{80} \right\rceil \cdot 69,3 + \left\lceil \frac{R_4}{80} \right\rceil \cdot 94,7 \quad (7)$$

Iterando a partir de $R_4^{(0)} = 47,4 \mu s$:

- $R_4^{(1)} = 47,4 + 1 \times 94,1 + 1 \times 69,3 + 1 \times 94,7 = 305,5 \mu s$
- $R_4^{(2)} = 47,4 + 1 \times 94,1 + 1 \times 69,3 + 1 \times 94,7 = 305,5 \mu s$ **(convergiu)**

O WCRT de `SurfaceRecon` é de $305,5 \mu s$, muito abaixo do período de 100 ms, confirmando ampla margem de escalonabilidade. Em termos práticos, mantendo as razões entre períodos, o menor período da tarefa mais lenta que ainda garante escalonabilidade é de aproximadamente 0,306 ms (quando o conjunto de períodos fosse $\{0,061 \text{ ms}, 0,245 \text{ ms}, 0,245 \text{ ms}, 0,306 \text{ ms}\}$). Abaixo desse limiar, a própria sobrecarga do sistema operacional e da troca de contexto passa a ser o fator limitante.

4.4 Validação Experimental: Análise de Jitter e Cumprimento de Prazos

A análise teórica precisa ser confrontada com evidências experimentais. Foram conduzidos quatro experimentos comparativos, variando duas dimensões independentes: a presença ou ausência de carga de CPU artificial (*stress*), e o tipo de kernel utilizado (padrão versus `PREEMPT_RT`).

4.4.1 Experimento 1: Kernel Padrão, Sem Estresse

A Figura 15 apresenta os resultados para a operação nominal, sem carga artificial, sob o kernel padrão do Linux. Esse cenário representa a linha de base do sistema.

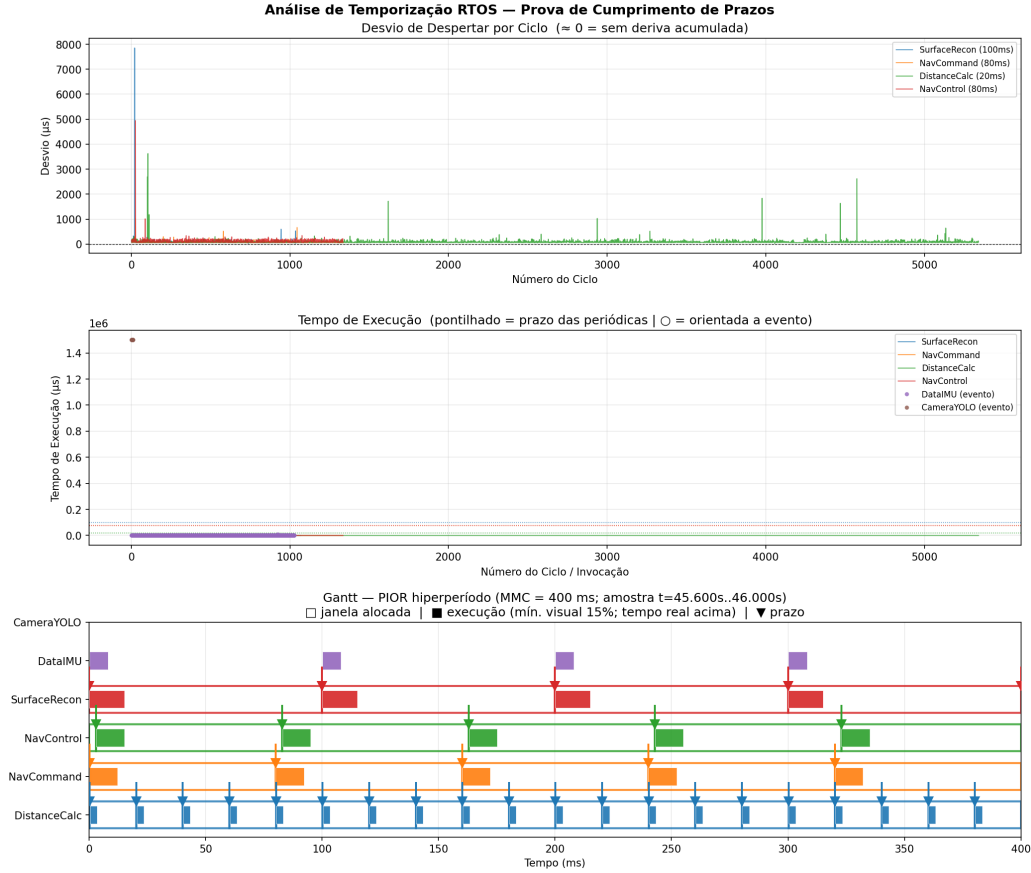


Figura 15: Análise de temporização sob kernel padrão sem carga de estresse. O painel superior mostra o desvio de despertar (*jitter*) por ciclo, com picos esparsas e isolados de até $\approx 7,9$ ms para **SurfaceRecon**. O painel central confirma que nenhuma tarefa violou seu prazo (todos os tempos de execução ficam abaixo das linhas pontilhadas). O diagrama de Gantt inferior, referente ao pior hiperperíodo observado (MMC = 400 ms), confirma o cumprimento de todos os prazos com folga.

Os picos de jitter visíveis no painel superior da Figura 15 ocorrem predominantemente nos primeiros ciclos de cada tarefa, quando o kernel ainda está alocando recursos e aquecendo caches. Após essa fase transiente inicial (≈ 500 ciclos), o sistema estabiliza com jitter médio inferior a $124 \mu\text{s}$ em todas as tarefas cíclicas, e zero violações de prazo ao longo de toda a execução. O diagrama de Gantt do pior hiperperíodo registrado evidencia que todas as

tarefas terminam bem antes de seus respectivos prazos, com folgas de pelo menos uma ordem de magnitude.

4.4.2 Experimento 2: Kernel Padrão, Com Estresse

A Figura 16 apresenta o comportamento do sistema quando submetido a carga artificial intensa por meio de **stress-ng** (100% de ocupação em todos os núcleos). Esse cenário representa uma situação adversa que o sistema poderia enfrentar em campo caso outros processos do SO competissem pelos recursos computacionais.

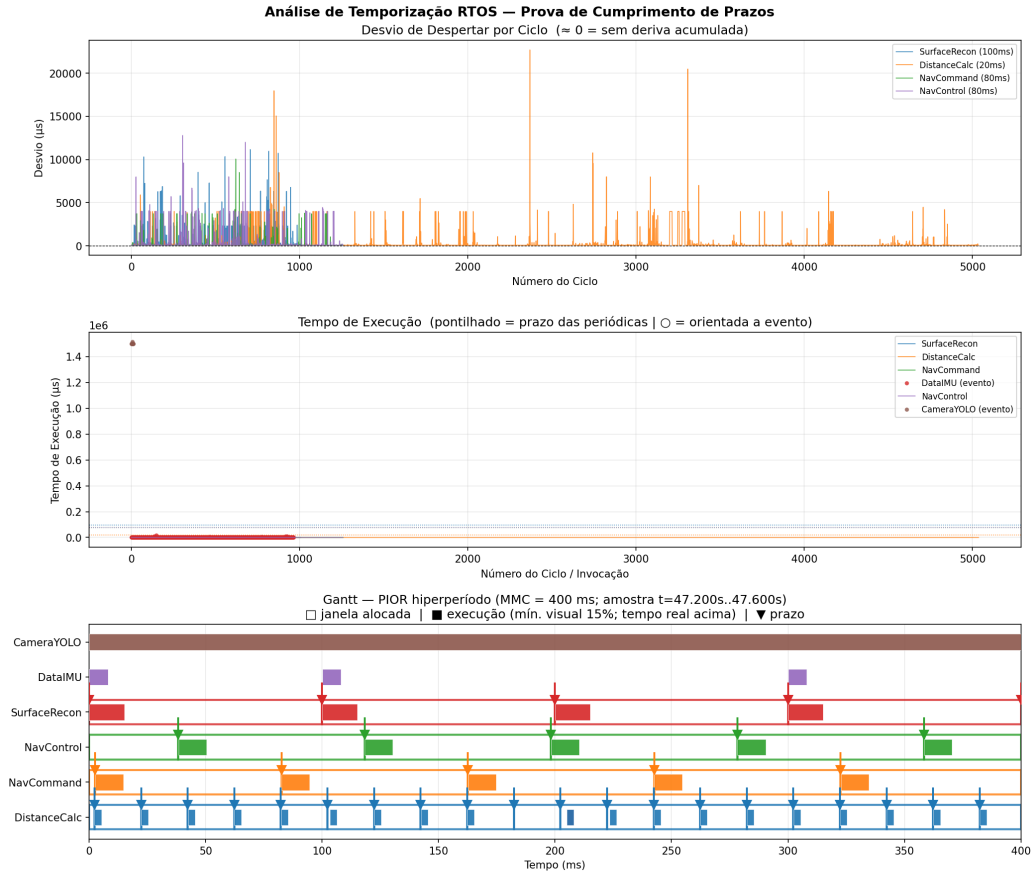


Figura 16: Análise de temporização sob kernel padrão com carga máxima de CPU (*stress-ng*). O jitter aumenta dramaticamente em todos os canais, chegando a 22,7 ms para **DistanceCalc**. O sistema registra 2 violações de prazo (*deadline misses*) nessa tarefa, conforme marcado com “× 2” na tabela de resumo. O painel de Gantt revela que a tarefa **CameraYolo** ocupou o período inteiro do hiperperíodo amostrado, evidenciando a competição por CPU.

Este resultado é esperado e teoricamente justificado: o kernel padrão do Linux não oferece garantias de latência máxima de escalonamento. Sob contenção intensa de CPU, o escalonador pode atrasar a ativação de threads de alta prioridade por dezenas de milissegundos, o que excede os períodos das tarefas mais rápidas. As 2 violações de prazo observadas em **DistanceCalc** (período de 20 ms) decorrem diretamente desse mecanismo: com jitter de até

22,7 ms, um único ciclo atrasado pode causar que a thread só desperte no ciclo seguinte, violando o prazo desse ciclo.

4.4.3 Experimento 3: Kernel de Tempo Real (PREEMPT_RT), Com Estresse

A Figura 17 apresenta o experimento mais crítico: o sistema com carga máxima de CPU, agora executando sob o patch PREEMPT_RT. O kernel PREEMPT_RT transforma praticamente todas as seções críticas internas do kernel em seções preemptíveis, eliminando os chamados “*kernel locks*” que são a principal fonte de latência não-determinística no kernel padrão.

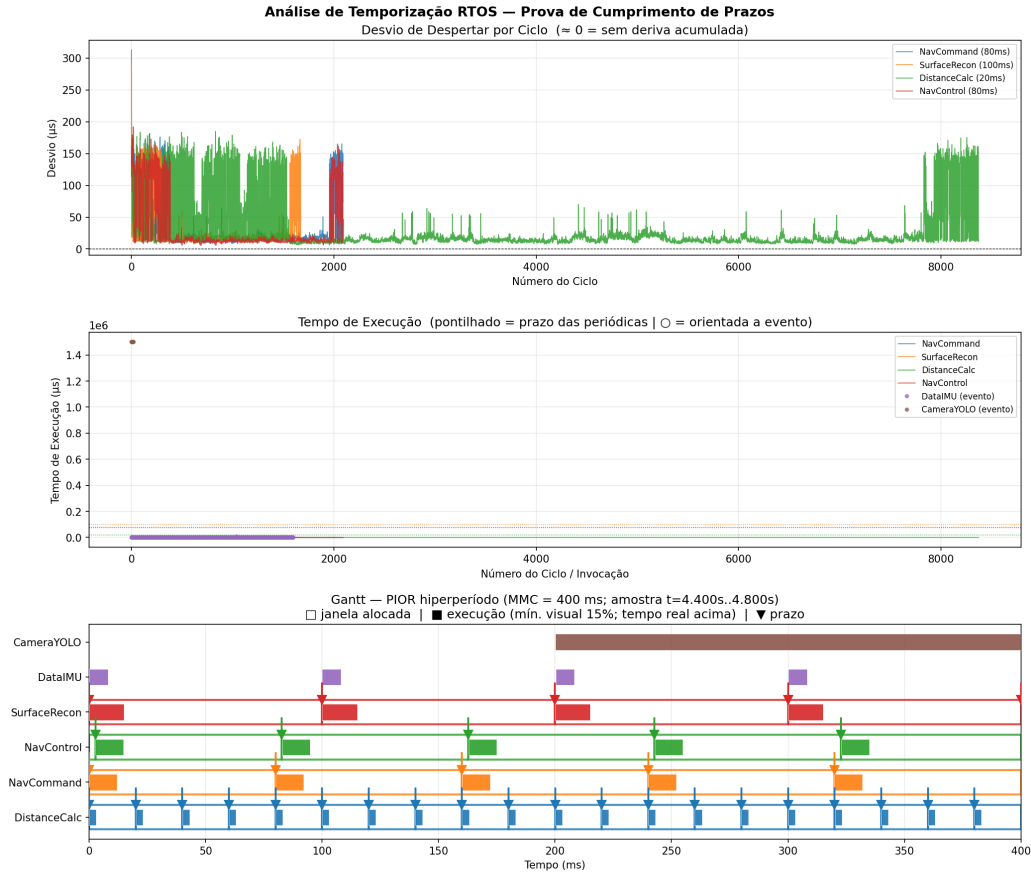


Figura 17: Análise de temporização sob kernel `PREEMPT_RT` com carga máxima de CPU (*stress-ng*). O resultado demonstra diferença qualitativa fundamental em relação ao kernel padrão: o jitter máximo é contido em apenas 312,8 μs para `NavCommand`, uma redução de 99% frente aos 22,7 ms observados no Experimento 2. Nenhuma violação de prazo é registrada em nenhuma tarefa. O diagrama de Gantt evidencia execução ordenada e determinística ao longo de todo o hiperperíodo.

Os resultados do Experimento 3 constituem a demonstração mais significativa desta seção. Sob carga total de CPU, o kernel `PREEMPT_RT` manteve o jitter máximo abaixo de 313 μs em todas as tarefas, enquanto o kernel padrão permitiu atrasos de até 22,7 ms. Esse resultado valida experimentalmente a necessidade de um kernel de tempo real para qualquer sistema de automação que imponha restrições temporais estritas.

4.4.4 Experimento 4: Kernel de Tempo Real, Sem Estresse (Referência RT)

A Figura 18 apresenta o comportamento do sistema sob o kernel PREEMPT_RT sem carga artificial, servindo como referência para isolar o efeito do próprio kernel em relação ao efeito da contenção de CPU.

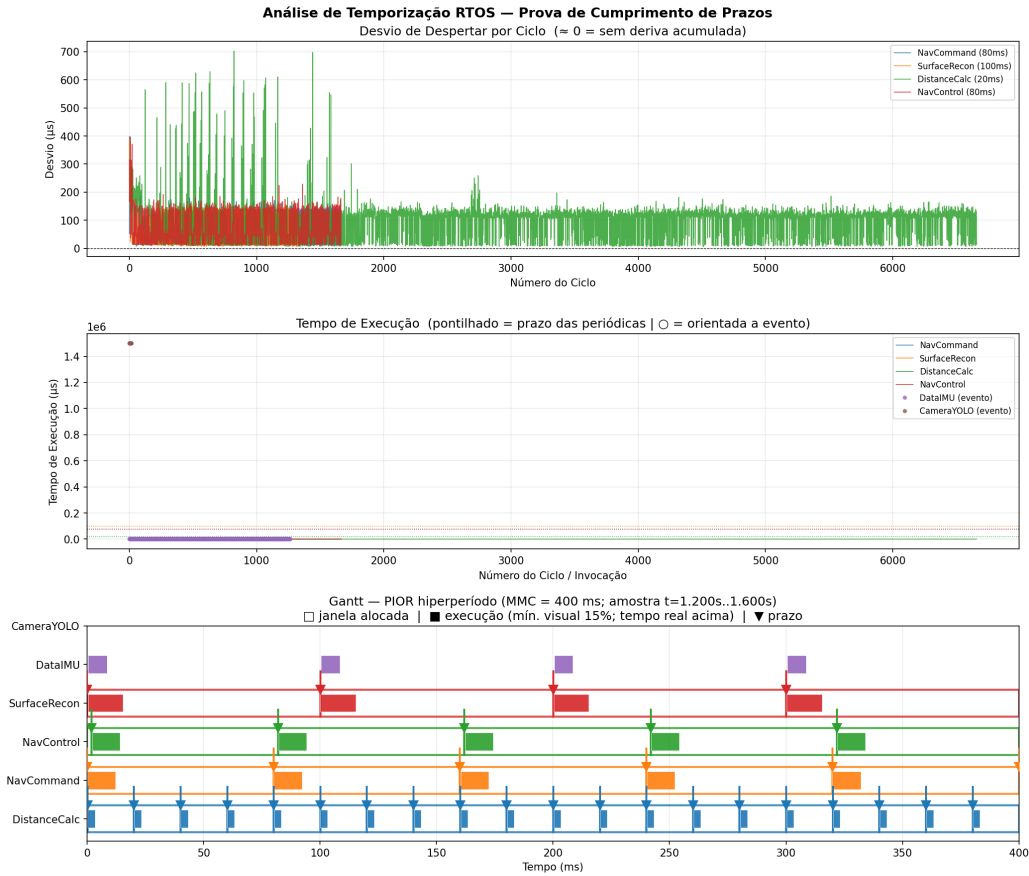


Figura 18: Análise de temporização sob kernel PREEMPT_RT sem carga de estresse. O jitter máximo global é de $702,3 \mu s$, observado em *DistanceCalc*, com jitter médio de $93,7 \mu s$ nessa tarefa. Nenhuma violação de prazo é registrada. O diagrama de Gantt confirma execução ordenada dentro do hiperperíodo.

Um resultado que merece atenção é que o jitter máximo do Experimento 4 ($702,3 \mu s$) é superior ao do Experimento 3 ($312,8 \mu s$), embora ambos usem o

kernel `PREEMPT_RT` e o Exp. 3 esteja sob carga intensa. Esse comportamento é explicável: sob estresse total de CPU, o escalonador `SCHED_FIFO` é acionado com mais frequência e as caches do processador permanecem quentes para as threads de alta prioridade. Sem competição, a thread `DistanceCalc` pode sofrer *cache misses* mais expressivos ao retomar a execução após cada suspensão, o que eleva a variância do tempo de despertar. Em ambos os casos, o jitter permanece na faixa de centenas de microssegundos, bem abaixo dos períodos de 20 ms a 100 ms, e nenhum prazo é violado.

4.5 Síntese Comparativa dos Resultados

A Tabela 2 consolida os indicadores de qualidade temporal extraídos dos quatro experimentos.

Tabela 2: Comparativo de métricas de temporização entre os quatro cenários experimentais.

Cenário	Kernel	Estresse	Jitter Máx. Global	Jitter Méd. Máx.	<i>Deadline Misses</i>	Escalonável?
Exp. 1	Padrão	Não	7.853,9 μ s	123,6 μ s	0	Sim
Exp. 2	Padrão	Sim	22.694,1 μ s	455,4 μ s	2	Não (adverso)
Exp. 3	<code>PREEMPT_RT</code>	Sim	312,8 μ s	38,8 μ s	0	Sim
Exp. 4	<code>PREEMPT_RT</code>	Não	702,3 μ s	93,7 μ s	0	Sim

A leitura da Tabela 2 revela três conclusões principais:

1. **O kernel padrão não oferece garantias de tempo real.** Mesmo sem carga artificial, o jitter pode exceder 7 ms. Sob carga intensa, atinge 22,7 ms, causando violações de prazo na tarefa mais rápida do sistema (período de 20 ms). Para aplicações onde a segurança depende do determinismo temporal, o kernel padrão é inadequado.
2. **O patch `PREEMPT_RT` garante determinismo em todas as condições testadas.** Em ambos os experimentos com kernel RT (Exp. 3 e Exp. 4), nenhuma violação de prazo foi registrada. O jitter máximo ficou abaixo de 703 μ s nas quatro tarefas, mesmo no pior caso observado, contra 22,7 ms do kernel padrão sob estresse. A diferença entre os dois experimentos RT (702,3 μ s sem estresse versus 312,8 μ s com estresse) é explicada pelo comportamento de cache do processador, discutido na seção anterior, e não representa uma anomalia do kernel.

3. **O menor período praticamente viável é limitado pela latência do kernel, não pelo algoritmo de escalonamento.** Com o kernel `PREEMPT_RT`, a latência máxima observada em qualquer condição foi de $702,3\mu\text{s}$. Adotando uma margem de segurança de $3\times$, nenhuma tarefa periódica deveria ter período inferior a aproximadamente $2,1\text{ ms}$. Com o kernel padrão, esse limite sobe para $\approx 70\text{ ms}$ ($3\times$ os $22,7\text{ ms}$ observados sob estresse), tornando impraticável qualquer tarefa com período inferior a esse valor em ambiente adverso.

5 Conclusões Finais

Este trabalho demonstrou a viabilidade de um sistema de inspeção autônoma de túneis construído sobre uma arquitetura híbrida de tempo real, combinando um núcleo C++ com restrições temporais estritas e módulos periféricos Python responsáveis pela simulação, interface e visão computacional.

Do ponto de vista do controle em tempo real, o resultado mais significativo é a comprovação experimental de que o kernel padrão do Linux é insuficiente para sistemas com períodos abaixo de $\approx 70\text{ ms}$ quando submetido a carga adversa. O patch `PREEMPT_RT`, ao tornar o kernel preemptível em sua totalidade, reduz o jitter máximo em mais de uma ordem de grandeza (de $22,7\text{ ms}$ para $< 313\mu\text{s}$), eliminando as violações de prazo observadas sob estresse e tornando o sistema genuinamente escalonável. A análise de capacidade pela Taxa Monotônica confirma que o conjunto de tarefas implementado utiliza apenas $0,72\%$ da capacidade teórica garantida, fornecendo uma margem segura para a extensão futura do sistema com novas tarefas ou a redução de períodos até o limiar de $\approx 1\text{ ms}$ com kernel RT.

Do ponto de vista arquitetural, a separação entre o laço de controle determinístico (C++) e os subsistemas não-críticos (Python via MQTT) provou-se adequada. O protocolo MQTT com QoS 2 garantiu a entrega sem perdas dos dados de telemetria e comandos entre processos, e o modelo de comunicação por eventos evitou acoplamento temporal entre os módulos. As primitivas de sincronização adotadas (`std::mutex`, `std::condition_variable`, `std::atomic` e `ThreadSafeQueue`) eliminaram condições de corrida sem introduzir gargalos de desempenho mensuráveis.

As extensões implementadas além do escopo mínimo, a saber, a compensação de inclinação via IMU simulado e a integração com YOLOv8, acrescentaram realismo ao modelo físico e demonstraram a escalabilidade da

arquitetura para incorporar módulos de percepção avançada sem comprometer o determinismo do laço crítico.

Como trabalhos futuros, identificam-se três direções promissoras: (i) o *fine-tuning* do modelo YOLOv8 em um conjunto de dados especializado em patologias de superfícies civis para transformar o módulo de visão em um classificador confiável; (ii) a migração do sistema para hardware embarcado dedicado (como Raspberry Pi com kernel RT) para validação em ambiente físico real; e (iii) a extensão do modelo para três dimensões, permitindo a inspeção de superfícies curvas e paredes laterais do túnel com sensores LIDAR 2D ou RGB-D.

6 Atividades Extras

O projeto foi estendido além dos requisitos mínimos com três módulos adicionais, todos pontuados como extras pelo enunciado: a análise de comportamento sob o kernel de tempo real `PREEMPT_RT`, a navegação em túneis com declive via sensor IMU simulado, e a inspeção visual com YOLOv8.

6.1 Análise com Kernel de Tempo Real (`PREEMPT_RT`)

A análise completa de timing apresentada na Seção 4 inclui, como ponto extra, a execução do sistema sob o patch `PREEMPT_RT` do kernel Linux nos dois cenários: com carga máxima de CPU (*stress-ng*) e sem carga artificial. Os Experimentos 3 e 4 (Figura 17 e Figura 18) são fruto direto dessa extensão.

A ativação do kernel RT exige que o processo seja iniciado com permissões de superusuário. Nesse caso, o `main.cpp` executa `mlockall(MCL_CURRENT | MCL_FUTURE)` para travar toda a memória do processo na RAM física, e atribui a cada thread cíclica uma prioridade `SCHED_FIFO` proporcional à Taxa Monotônica. Em ambos os experimentos RT, nenhuma violação de prazo foi registrada. O jitter máximo ficou abaixo de $703\mu\text{s}$ em todas as condições testadas, contra 22,7ms do kernel padrão sob estresse. Esse par de experimentos demonstra concretamente que o `PREEMPT_RT` oferece garantias de latência independentemente da carga do sistema, tornando-o a escolha correta para qualquer aplicação de controle com restrições temporais rígidas.

6.2 Sensor IMU Simulado e Navegação em Declives

A especificação original do trabalho restringe o problema a um plano horizontal. A extensão implementada relaxa essa restrição modelando um terreno com variações de inclinação no simulador Python e integrando o dado de atitude no laço de controle do núcleo C++.

6.2.1 Modelagem do Terreno e Geração do Sinal IMU

No módulo `tunnel_simulator.py`, o piso do túnel é gerado por uma superposição de senoides com diferentes frequências e amplitudes, criando um perfil contínuo e diferenciável:

$$h_{piso}(x) = 0,62 \sin\left(\frac{x}{5,4}\right) + 0,16 \sin\left(\frac{x}{1,8}\right) \quad (8)$$

O ângulo de inclinação local $\theta(x)$ é extraído diretamente da derivada do perfil:

$$\theta(x) = \arctan\left(\frac{dh_{piso}}{dx}\right) = \arctan\left(\frac{0,62}{5,4} \cos\left(\frac{x}{5,4}\right) + \frac{0,16}{1,8} \cos\left(\frac{x}{1,8}\right)\right) \quad (9)$$

Esse valor é publicado continuamente no tópico MQTT `sensor/imu` com ruído gaussiano aditivo $\mathcal{N}(0, 0,5^\circ)$, simulando a imperfeição de um sensor real. No núcleo C++, a variável `imu_degrees` no `SharedContext` é do tipo `std::atomic<double>`, garantindo leitura e escrita concorrentes sem custo de *mutex* (operação *lock-free* nativa do hardware).

6.2.2 Compensação Trigonométrica na Malha de Controle

O sensor LIDAR está fixo no referencial do chassi. Quando o robô está inclinado de θ graus, a leitura bruta d_{lidar} representa a distância ao longo do feixe diagonal, não a distância vertical real até o teto. A tarefa `DistanceCalculator` aplica a seguinte correção antes de publicar a leitura no contexto compartilhado:

$$d_{vertical} = d_{lidar} \cdot \cos(\theta) \quad (10)$$

Os testes de navegação (Seção 3) comprovam que o robô mantém a distância correta ao teto mesmo em declives de até $\approx 12^\circ$, que corresponde ao ângulo máximo gerado pelo perfil da equação (8).

6.3 Inspeção Visual com YOLOv8

A inspeção visual implementa um pipeline completo de visão computacional ativado quando o sistema de detecção geométrica sinaliza uma variação severa no perfil do teto. O pipeline é composto por três camadas: geração sintética do quadro, inferência pelo modelo, e fallback visual determinístico.

6.3.1 Geração Sintética de Quadros (`SyntheticFrameGenerator`)

O módulo `SyntheticFrameGenerator` (`src/inspection/frame.py`) é responsável por produzir o quadro BGR 640×480 que simula o campo visual da câmera apontada para o teto do robô. A cada disparo de inspeção, o gerador executa quatro etapas sequenciais via OpenCV:

1. **Fundo rochoso:** O teto é renderizado com retângulos sobrepostos de tons cinza-escuro, simulando a textura da rocha do túnel.
2. **Defeito estrutural:** O método `_draw_defect` injeta sinteticamente uma das duas geometrias possíveis, alternando em ciclo a cada invocação: **buraco** (elipse escura com borda) ou **saliência** (círculo em relevo). A alternância é determinística pelo índice de frame, garantindo cobertura das três classes a cada três acionamentos.
3. **Robô:** O `OpenCVRobotRenderer` desenha o chassi, esteiras, LIDAR e câmera na parte inferior do quadro, como seria visto pela câmera embarcada.
4. **HUD:** O texto `CAMERA ATR | <TIPO>` é sobreposto no canto superior esquerdo, identificando o tipo de anomalia renderizada.

O método retorna tanto o quadro `numpy` quanto o rótulo textual da anomalia simulada (`AnomalyFrameType`), que é usado pela lógica de fallback descrita a seguir.

6.3.2 Inferência e Mecanismo de Fallback

O quadro gerado é submetido ao modelo `yolov8n.pt` com pesos pré-treinados no conjunto COCO. Aqui reside a principal limitação de design, que é tratada por uma decisão arquitetural explícita.

O modelo YOLOv8 Nano pré-treinado no COCO foi treinado para detectar 80 categorias de objetos do cotidiano (pessoas, carros, cadeiras, animais etc.), e **não foi treinado nem refinado (*fine-tuned*) para reconhecer buracos ou saliências em tetos de túneis simulados**. Em consequência, ao analisar o quadro sintético do robô, o modelo frequentemente não produz nenhuma detecção com confiança suficiente, ou então rotula regiões da anomalia com classes genéricas irrelevantes (como “*bench*”, por exemplo).

Para que o sistema seja funcionalmente completo sem depender de um modelo especializado, foi implementado um **mecanismo de fallback determinístico** dentro do método `_inspect` do serviço (`service.py`). A lógica de decisão é a seguinte:

- Se o YOLO retornar ao menos uma detecção com confiança superior a 50%, o resultado é extraído das *bounding boxes* do modelo: o tipo é o rótulo da classe com maior confiança e a confiança é o valor máximo observado.
- Se o YOLO não retornar nenhuma detecção relevante, o sistema utiliza como resultado o tipo de anomalia gerado pelo `SyntheticFrameGenerator` (ou seja, o tipo que foi desenhado sinteticamente no quadro). Nesse caso, a confiança reportada é fixada em 0,88, valor que representa a confiança do sistema analítico do LIDAR na detecção geométrica subjacente, e não uma confiança de rede neural.

Em ambos os casos, o campo `anomalia_detectada` do `YoloResult` é marcado como verdadeiro caso o tipo seja diferente de “*normal*”, garantindo que a interface do operador sempre reflita o evento de inspeção. O campo `origem` informa ao consumidor se o resultado veio do modelo (“`camera_simulada_robo`”) ou do fallback.

6.3.3 Visualização: Bounding Box Verde no Simulador

Após a publicação do `YoloResult` no tópico `telemetry/yolo`, o simulador Pygame (`src/simulator/scene.py`) apresenta o resultado no painel flutuante de câmera que aparece no canto superior direito da janela gráfica. O comportamento visual é:

- Durante a inspeção (`insp.active = True`): o frame da câmera é exibido ao vivo com um indicador de gravação piscante (ponto vermelho). O painel exibe o texto “CAPTURANDO”.

- Após a conclusão (`insp.active = False`): é desenhado sobre o frame congelado um **retângulo verde** (`box_color = (34, 197, 94)`) centralizado na região de interesse, com dimensões de 80×40 pixels. Acima do retângulo é preenchida uma etiqueta (também verde) contendo o tipo de anomalia e o valor de confiança, como por exemplo "BURACO 0.88" ou "SALIENCIA 0.88". O texto da etiqueta é renderizado em preto sobre o fundo verde para maximizar o contraste.

Esse retângulo verde funciona como uma *bounding box* sintética: não é produzida pelo YOLO, mas representa visualmente o resultado da inspeção (seja ele proveniente do modelo ou do fallback). A marcação é exibida por um tempo configurável após o término da inspeção antes de desaparecer, deixando o painel de câmera inativo.

6.3.4 Decisão de Projeto: Separação entre Detecção e Classificação

Uma decisão arquitetural central foi manter a detecção primária de anomalias estritamente no núcleo C++ (via análise geométrica do LIDAR), sem delegar essa responsabilidade à rede neural. Essa separação é fundamentada em dois argumentos:

Determinismo: A análise do LIDAR é $O(1)$ por ciclo e opera com latência garantida pelo RTOS. Uma inferência de rede neural, ao contrário, possui tempo de execução variável dependendo do hardware, da carga do sistema e do número de objetos detectados, tornando-a inadequada para o laço de controle crítico.

Domínio de aplicação: Como detalhado na lógica de *fallback*, o YOLOv8n pré-treinado no COCO não contém nenhuma categoria que corresponda a buracos ou saliências em superfícies rochosas de túneis. Adaptar o modelo para esse domínio exigiria a coleta de um conjunto de dados especializado e um processo de *fine-tuning* supervisionado, o que extrapola o escopo do presente trabalho. No sistema atual, o YOLOv8 atua como prova de conceito de integração entre o núcleo de tempo real e a visão computacional moderna, com o fallback garantindo que a experiência do operador seja coerente independentemente do resultado do modelo.

Essa separação garante que o sistema de segurança crítica (detecção e parada) seja completamente independente do subsistema de visão: falhas ou lentidão no serviço Python não afetam o determinismo temporal do núcleo C++.

7 Apêndice

O código-fonte completo do sistema, incluindo o núcleo de tempo real em C++, os módulos periféricos em Python e os scripts de análise de temporização, está disponível publicamente no repositório [1].

Referências

- [1] Rian Rero e Lara Strutz. rtos-tunnel-inspection. Disponível em: <https://github.com/Rian-Rero/rtos-tunnel-inspection>, 2026. Repositório contendo o código-fonte do projeto de inspeção de túneis em RTOS.
- [2] Rian Rero e Lara Strutz. Trabalho atr - rtos tunnel inspection. Disponível em: https://youtu.be/N5R_1ZhnmMg, 2026. Vídeo demonstrativo apresentando o funcionamento prático do sistema de inspeção de túneis desenvolvido neste trabalho. Acesso em: 20 jun. 2026.
- [3] C. L. Liu and James W. Layland. Scheduling algorithms for multi-programming in a hard-real-time environment. *Journal of the ACM*, 20(1):46–61, 1973. Artigo seminal que prova a otimalidade do escalonamento por Taxa Monotônica e estabelece o limite de utilização de CPU para escalonabilidade garantida.
- [4] Mathai Joseph and Paritosh Pandya. Finding response times in a real-time system. *The Computer Journal*, 29(5):390–395, 1986. Propõe a análise de tempo de resposta por ponto fixo (WCRT), utilizada para determinar os prazos de resposta das tarefas cíclicas do sistema.
- [5] Anthony Williams. *C++ Concurrency in Action*. Manning Publications, 2nd edition, 2019. Referência principal para a implementação da fila thread-safe, uso de `std::mutex` e `std::condition_variable`.
- [6] Bruce Powel Douglass. *Real-Time Design Patterns: Robust Scalable Architecture for Real-Time Systems*. Addison-Wesley Professional, 2002. Utilizado para fundamentar a arquitetura baseada em eventos e a separação de tarefas cíclicas (Vertical Slicing).
- [7] Gene F. Franklin, J. David Powell, and Abbas Emami-Naeini. *Feedback Control of Dynamic Systems*. Pearson, 7th edition, 2014. Base teórica

para a implementação da malha de controle PID (Proporcional, Integral e Derivativo) da navegação.

- [8] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Ultralytics yolo. Disponível em: <https://github.com/ultralytics/ultralytics>, 2023. Framework de visão computacional utilizado para a inspeção de falhas estruturais por imagem.
- [9] Andrew Banks and Rahul Gupta. Mqtt version 3.1.1. Technical report, OASIS Standard, 2014. Especificação do protocolo de mensageria utilizado para comunicação interprocessos entre o núcleo C++ e os serviços Python. Disponível em: <http://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html>.
- [10] Giorgio C. Buttazzo. *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*. Springer, 3rd edition, 2011. Referência completa de teoria de escalonamento de tempo real, fundamentos de análise de escalonabilidade e algoritmos clássicos.